

Multimodal visual understand + Depth Camera Distance Question Answering(Text Version)

Multimodal visual understand + Depth Camera Distance Question Answering(Text Version)

1. Course Content
2. Preparation
 - 2.1 Content Description
3. Running the Example
 - 3.1 Starting the Program
 - 3.2 Test Cases
 - 3.2.1 Case 1: "Please tell me the distance between the fan in front of you and you."
4. Source Code Parsing
 - 4.1 Example 1

1. Course Content

1. Learn to use the robot's visual feedback to determine the distance to an object.
2. Analyze newly discovered key source code.

2. Preparation

2.1 Content Description

This course uses the Raspberry Pi 5 as an example. For Raspberry Pi and Jetson Nano boards, you need to open a terminal on the host computer and enter the command to enter the Docker container. Once inside the Docker container, enter the commands mentioned in this course in the terminal. For instructions on entering the Docker container from the host computer, refer to **[01. Robot Configuration and Operation Guide] -- [5.Enter Docker (For JETSON Nano and RPi 5)]**. For Orin boards, simply open a terminal and enter the commands mentioned in this course.

 This example uses `model: "qwen/qwen2.5-v1-72b-instruct:free", "qwen-v1-latest"`

 The responses from the large model may not be exactly the same for the same test command and may differ slightly from the screenshots in the tutorial. To increase or decrease the diversity of the large model's responses, refer to the section on configuring the decision-making large model parameters in the **[04.AI Large Model Development] -- [5.Deploy the RAG knowledge base]**.

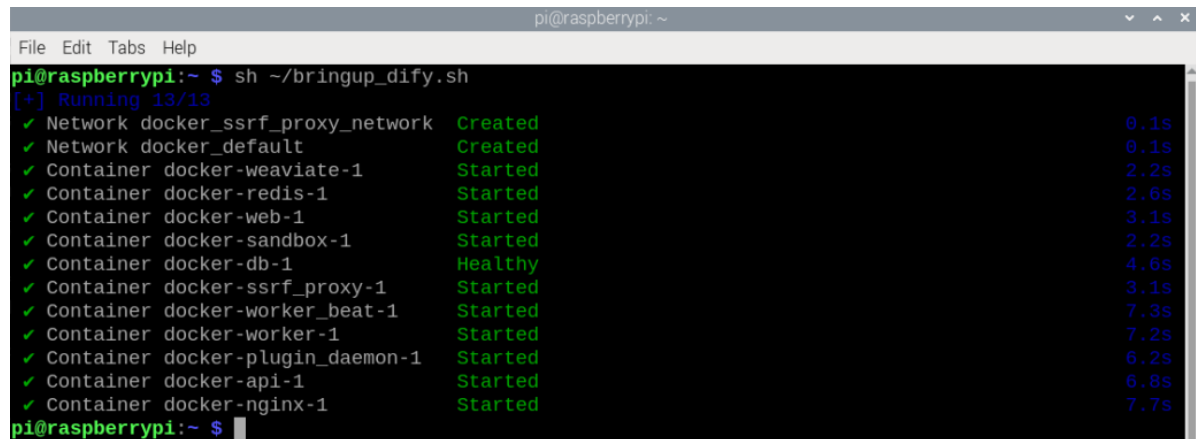
 It is recommended that you first try the previous visual example. This example adds voice functionality to the singleton example. The functionality is largely the same, so I will not further debug the program or describe the results in detail.

3. Running the Example

3.1 Starting the Program

Open the host terminal and start the Dify environment:

```
sh ~/bringup_dify.sh
```



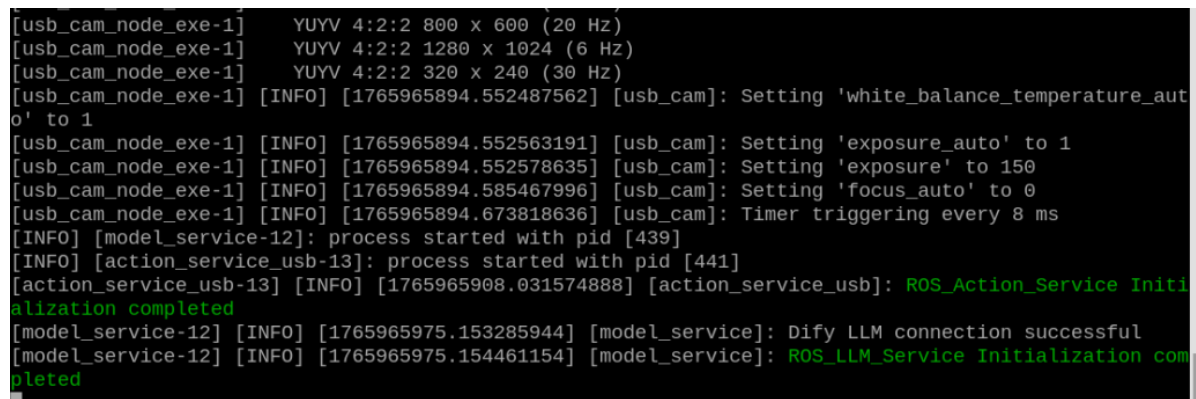
```
pi@raspberrypi: ~  
File Edit Tabs Help  
pi@raspberrypi:~$ sh ~/bringup_dify.sh  
[+] Running 13/13  
✔ Network docker_ssrf_proxy_network Created 0.1s  
✔ Network docker_default Created 0.1s  
✔ Container docker-weaviate-1 Started 2.2s  
✔ Container docker-redis-1 Started 2.6s  
✔ Container docker-web-1 Started 3.1s  
✔ Container docker-sandbox-1 Started 2.2s  
✔ Container docker-db-1 Healthy 4.6s  
✔ Container docker-ssrf_proxy-1 Started 3.1s  
✔ Container docker-worker_beat-1 Started 7.3s  
✔ Container docker-worker-1 Started 7.2s  
✔ Container docker-plugin_daemon-1 Started 6.2s  
✔ Container docker-api-1 Started 6.8s  
✔ Container docker-nginx-1 Started 7.7s  
pi@raspberrypi:~$
```

For Raspberry Pi 5 and Jetson Nano controllers, you must first enter the Docker container. For RDKX5 and Orin board, this is not necessary.

Open a terminal in Docker and enter the following command:

```
ros2 launch multi_brains llm_agent_control.launch.py text_chat_mode:=True
```

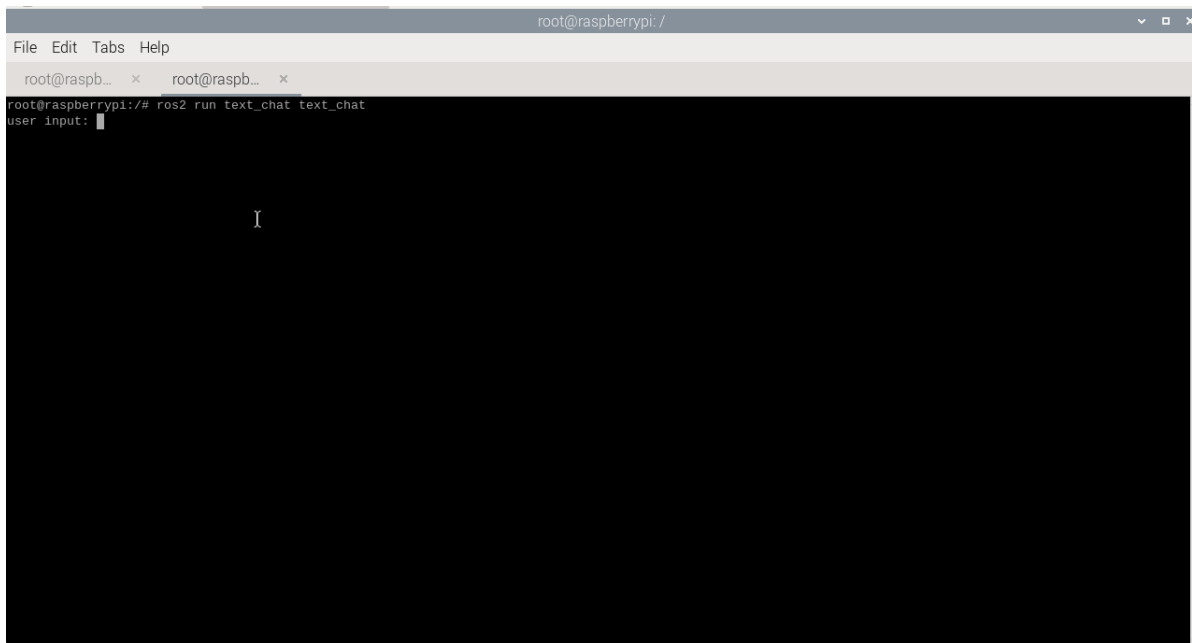
After initialization, the following content will be displayed:



```
[usb_cam_node_exe-1] YUYV 4:2:2 800 x 600 (20 Hz)  
[usb_cam_node_exe-1] YUYV 4:2:2 1280 x 1024 (6 Hz)  
[usb_cam_node_exe-1] YUYV 4:2:2 320 x 240 (30 Hz)  
[usb_cam_node_exe-1] [INFO] [1765965894.552487562] [usb_cam]: Setting 'white_balance_temperature_auto' to 1  
[usb_cam_node_exe-1] [INFO] [1765965894.552563191] [usb_cam]: Setting 'exposure_auto' to 1  
[usb_cam_node_exe-1] [INFO] [1765965894.552578635] [usb_cam]: Setting 'exposure' to 150  
[usb_cam_node_exe-1] [INFO] [1765965894.585467996] [usb_cam]: Setting 'focus_auto' to 0  
[usb_cam_node_exe-1] [INFO] [1765965894.673818636] [usb_cam]: Timer triggering every 8 ms  
[INFO] [model_service-12]: process started with pid [439]  
[INFO] [action_service_usb-13]: process started with pid [441]  
[action_service_usb-13] [INFO] [1765965908.031574888] [action_service_usb]: ROS_Action_Service Initialization completed  
[model_service-12] [INFO] [1765965975.153285944] [model_service]: Dify LLM connection successful  
[model_service-12] [INFO] [1765965975.154461154] [model_service]: ROS_LLM_Service Initialization completed
```

Open the same Docker container in multiple terminals and start it.

```
ros2 run text_chat text_chat
```



3.2 Test Cases

Here are two reference test cases; users can create their own test commands.

- Please tell me the distance between the object in front of you and you.

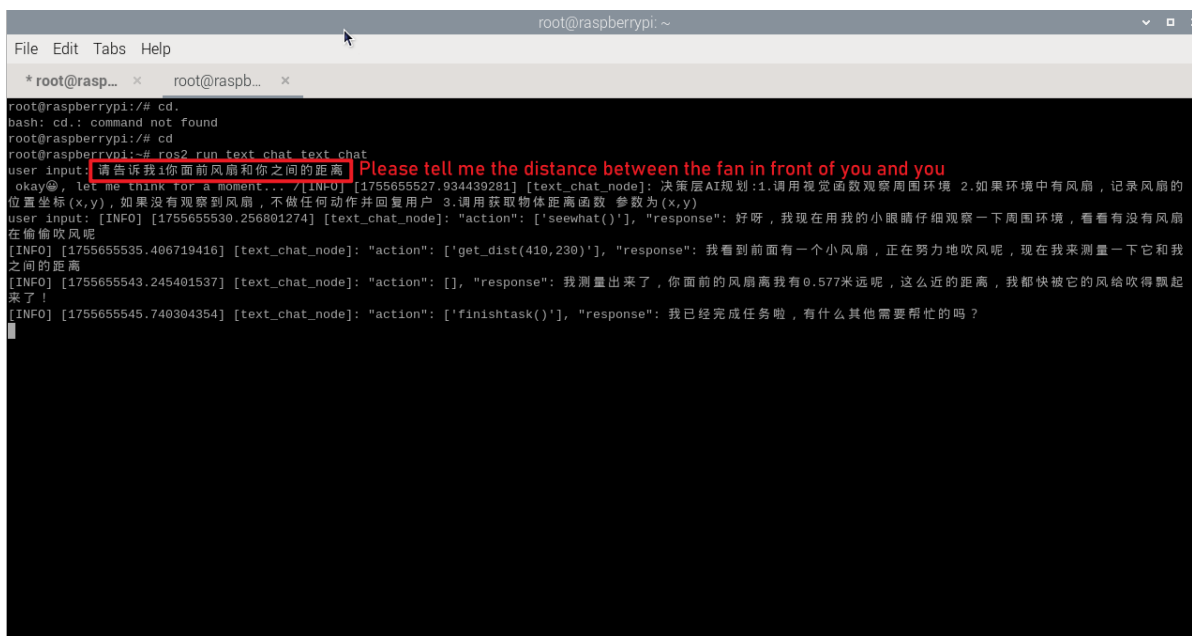
Example: Please tell me the distance between the fan in front of you and you.

⚠ Please do not end the text with a period or any other characters!

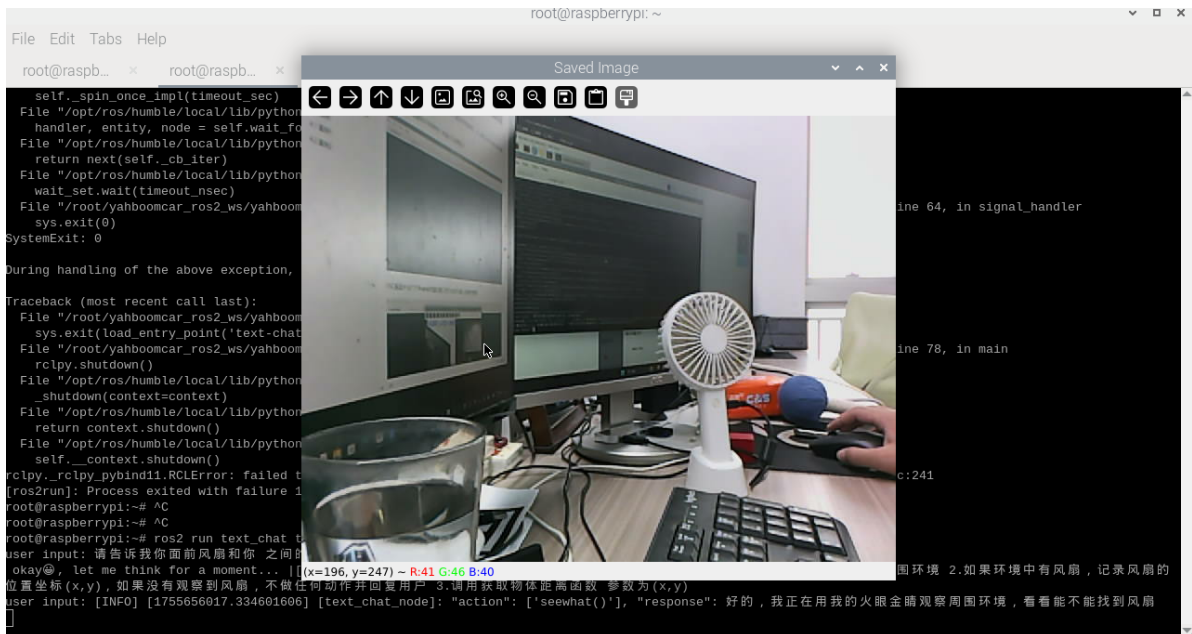
3.2.1 Case 1: "Please tell me the distance between the fan in front of you and you."

Type "Please tell me the distance between the fan in front of you and you." The terminal prints the following information.

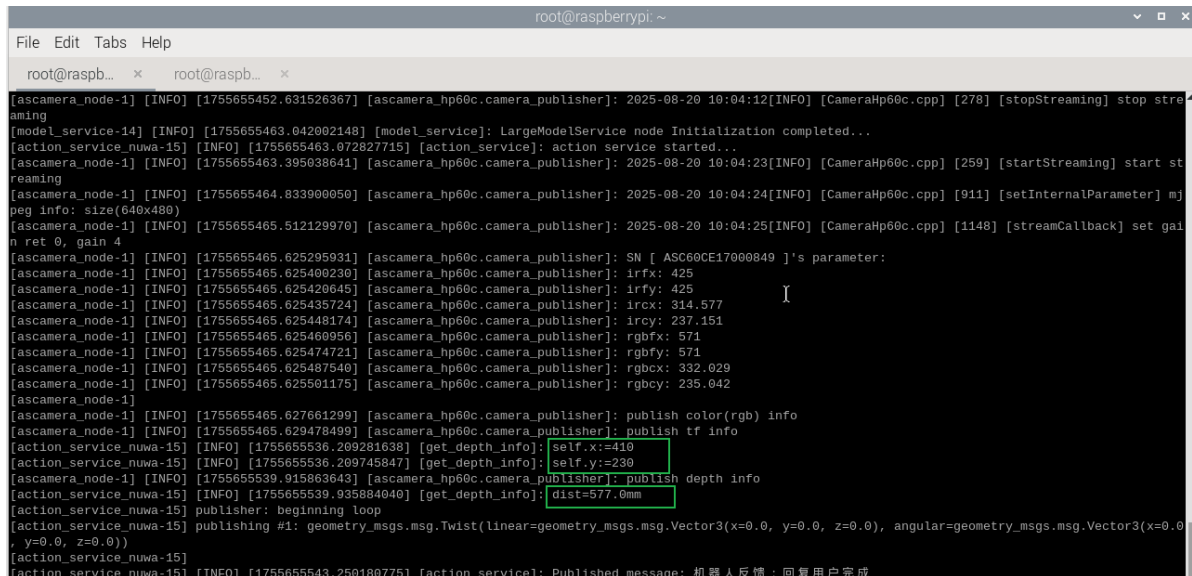
You can see that the terminal responds with a distance of 0.577 meters. **(Note: If the feedback is 0.0 meters, it may be because the object being recognized is too small, resulting in inaccurate center coordinates from the large vision model. This can be resolved by changing the vision model and the recognized object.)**



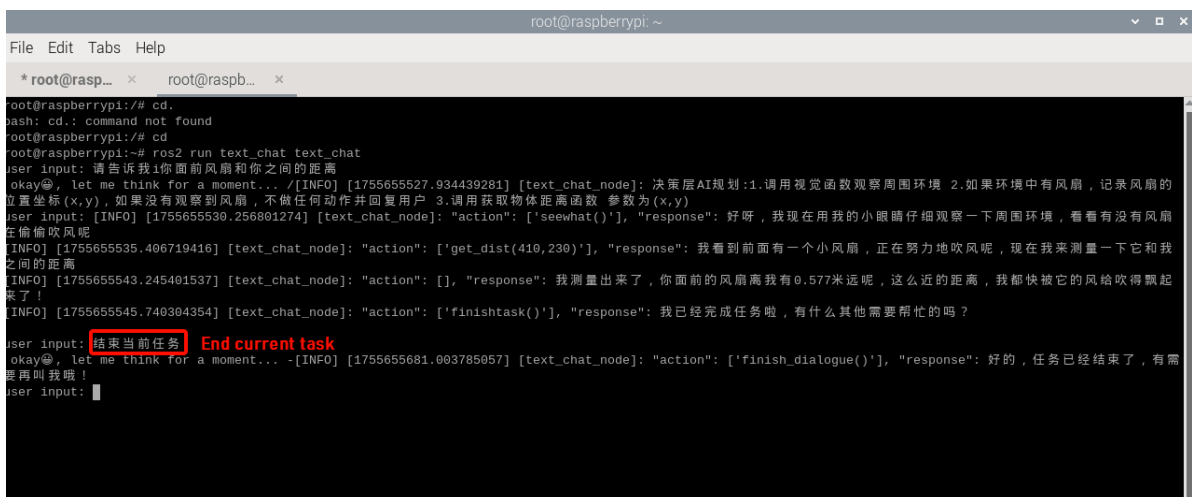
A window titled **frame** will open on the VNC screen and automatically close after 4 seconds.



The other terminal will print the object's center coordinates and distance information.



After completing a task, the robot enters a waiting state. Pressing the **ENTER** key allows you to continue the conversation. The command you entered is directly passed to the execution layer model, and all conversation history is retained. You can enter the **"End current task"** command again to terminate the current task cycle and start a new one.



```
root@raspberrypi: ~
File Edit Tabs Help
root@raspb... x root@raspb... x
[ascamera_node-1] [INFO] [1755655452.631526367] [ascamera_hp60c.camera_publisher]: 2025-08-20 10:04:12[INFO] [CameraHp60c.cpp] [278] [stopStreaming] stop streaming
[model_service-14] [INFO] [1755655463.042002148] [model_service]: LargeModelService node Initialization completed...
[action_service_nuwa-15] [INFO] [1755655463.072827715] [action_service]: action service started...
[ascamera_node-1] [INFO] [1755655463.395038641] [ascamera_hp60c.camera_publisher]: 2025-08-20 10:04:23[INFO] [CameraHp60c.cpp] [259] [startStreaming] start streaming
[ascamera_node-1] [INFO] [1755655464.833900050] [ascamera_hp60c.camera_publisher]: 2025-08-20 10:04:24[INFO] [CameraHp60c.cpp] [911] [setInternalParameter] mjpeg info: size(640x480)
[ascamera_node-1] [INFO] [1755655465.512129970] [ascamera_hp60c.camera_publisher]: 2025-08-20 10:04:25[INFO] [CameraHp60c.cpp] [1148] [streamCallback] set gain ret 0, gain 4
[ascamera_node-1] [INFO] [1755655465.625295931] [ascamera_hp60c.camera_publisher]: SN [ ASC60CE17000849 ]'s parameter:
[ascamera_node-1] [INFO] [1755655465.625400230] [ascamera_hp60c.camera_publisher]: irfx: 425
[ascamera_node-1] [INFO] [1755655465.625420645] [ascamera_hp60c.camera_publisher]: irfy: 425
[ascamera_node-1] [INFO] [1755655465.625435724] [ascamera_hp60c.camera_publisher]: ircx: 314.577
[ascamera_node-1] [INFO] [1755655465.625448174] [ascamera_hp60c.camera_publisher]: ircy: 237.151
[ascamera_node-1] [INFO] [1755655465.625460956] [ascamera_hp60c.camera_publisher]: rgbfx: 571
[ascamera_node-1] [INFO] [1755655465.625474721] [ascamera_hp60c.camera_publisher]: rgbfy: 571
[ascamera_node-1] [INFO] [1755655465.625487540] [ascamera_hp60c.camera_publisher]: rgbcx: 332.029
[ascamera_node-1] [INFO] [1755655465.625501175] [ascamera_hp60c.camera_publisher]: rgbcy: 235.042
[ascamera_node-1] [INFO] [1755655465.627661299] [ascamera_hp60c.camera_publisher]: publish color(rgb) info
[ascamera_node-1] [INFO] [1755655465.629478499] [ascamera_hp60c.camera_publisher]: publish tf info
[action_service_nuwa-15] [INFO] [1755655536.209281638] [get_depth_info]: self.x:=410
[action_service_nuwa-15] [INFO] [1755655536.209745847] [get_depth_info]: self.y:=230
[ascamera_node-1] [INFO] [1755655539.915863643] [ascamera_hp60c.camera_publisher]: publish depth info
[action_service_nuwa-15] [INFO] [1755655539.935884040] [get_depth_info]: dist=577.0mm
[action_service_nuwa-15] publisher: beginning loop
[action_service_nuwa-15] publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0))
[action_service_nuwa-15] [INFO] [1755655543.250180775] [action_service]: Published message: 机器人反馈：回复用户完成
[model_service-14] [INFO] [1755655681.008500073] [model_service]: The current instruction cycle has ended
[action_service_nuwa-15] [INFO] [1755655681.013758046] [action_service]: Published message: finish
```

4. Source Code Parsing

Source code located at:

Jetson Orin Nano:

```
#NUWA camera user
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/largemodel/largemodel/action_service_nuwa.py
#USB camera user
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/largemodel/largemodel/action_service_usb.py
```

jetson Nano, Raspberry Pi:

You need to enter Docker first.

```
#NUWA Camera User
/root/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_nuwa.py
#USB Camera User
/root/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_usb.py
```

RDKX5:

```
#NUWA Camera User
/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_nuwa.py
#USB Camera User
/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_usb.py
```

4.1 Example 1

action_service.py Program:

Example 1 uses the **seewhat** and **get_dist(self,x,y)** methods in the **CustomActionServer** class.

- The **seewhat** function primarily obtains the color image from the depth camera.
- The **get_dist(self,x,y)** function performs color tracking.
- **dist_callback()** receives the distance callback function

Here we mainly explain the **get_dist(self,x,y)** function. This function requires input parameters x and y, representing the center coordinates of the distance to be obtained.

Subscribe to the feedback distance

```
self.dist_sub =  
self.create_subscription(Int32, 'dist_topic', self.dist_callback, 10)
```

Deep Distance Question and Answer Subprocess

```
在偷偷吹风呢  
[INFO] [175565535.406719416] [text_chat_node]: "action": ['get_dist(410,230)'], "response": 我看到前面有一个小风扇，正在努力地吹风呢，现在我来测量一下它和我之间的距离
```

```
# Start the Deep Distance Question and Answer subprocess  
process_1 = subprocess.Popen(['ros2', 'run', 'yahboomcar_voice_ctrl_depth',  
['voice_get_dist', '--ros-args', '-p', f'x:={x1}', '-p', f'y:={y1}']])
```

The startup program source code path is:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_voice_ctrl_depth/yahboomcar_voice  
_ctrl_depth/voice_get_dist.py
```

```
def get_dist(self,x,y):  
    self.get_dist_future = Future() #Reset the Future object  
    x1 = int(x)  
    y1 = int(y)  
    process_1 = subprocess.Popen(['ros2', 'run', 'yahboomcar_voice_ctrl_depth',  
'voice_get_dist', '--ros-args', '-p', f'x:={x1}', '-p', f'y:={y1}'])  
    while not self.get_dist_future.done():  
        if self.interrupt_flag:  
            break  
        time.sleep(0.1)  
  
    self.kill_process_tree(process_1.pid)  
    self.cancel()
```

Wait for the callback function to feedback the distance and send the future.done signal. Then the `while not self.get_dist_future.done()` in the **dist_callback** function will exit the blocking state. Then the **kill_process_tree** method will be called to recursively kill the process tree of the child process. Finally, the status of the execution action will be fed back to the execution layer model.

